UNIVERSITÀ DI PISA

University of Pisa

Competitive Programming and Contests

Academic Year 2025–2026

HANDS-ON REPORT

Solution to Hands-On 1

Recursive Traversals of a Binary Tree in Rust

This report presents the solutions for the first hands-on assignment of the course. The work focuses on recursive binary-tree traversals in Rust, including binary-search-tree validation and the maximum path sum between two leaves.

Name Oleksiy Nedobiychuk
Matricola 597455
Course Competitive Programming and Contests
Date July 13, 2026

Department of Computer Science, University of Pisa

Introduction

Here, I present the solutions to the first and second hands-on exercises. The first solution (Exercise 1) uses two approaches: a recursive traversal with value boundaries and an iterative approach called Morris traversal. The second solution (Exercise 2) uses only a recursive approach, since the problem is a post-order dynamic programming problem, which requires O(H) space complexity, where H is the height of the tree. The iterative approach would require O(H) space complexity, where H is the height of the tree.

Exercise 1: Binary Search Tree Validation

Firstly, I implemented a recursive approach to validate if a binary tree is a binary search tree (BST). The algorithm checks if the value of each node is within a valid range defined by its ancestors. The time complexity of this approach is O(N), where N is the number of nodes in the tree, and the space complexity is O(H), where H is the height of the tree. After a search for a better solution, I found the Morris traversal algorithm, which allows us to validate the BST in O(N) time and O(1) space complexity. This method modifies the tree structure temporarily to avoid using additional space for recursion or a stack.

Exercise 2: Maximum Path Sum Between Two Leaves

For the second exercise, I implemented a recursive approach to find the maximum path sum between two leaves in a binary tree. The algorithm computes the maximum path sum for each node by considering the maximum path sums of its left and right subtrees. The time complexity of this approach is O(N), where N is the number of nodes in the tree, and the space complexity is O(H), where H is the height of the tree, due to the recursive call stack.

Testing

To validate the implementation, I wrote a set of unit tests in Rust. The goal of the test suite is not only to check correctness on standard examples, but also to cover edge cases that are easy to get wrong in recursive tree algorithms.

For the BST validation problem, the tests compare the recursive method and the Morris traversal on the same input trees. In particular, I included:

- a single-node tree, which must always be a valid BST;
- valid sparse trees and skewed trees, to check boundary conditions and unbalanced shapes;
- trees with immediate violations, such as a right child smaller than its parent;
- trees with deep violations, where a node is locally correct with respect to its parent but invalid with respect to an ancestor;
- trees containing duplicate keys, which must be rejected by a strict BST definition;
- trees using boundary values such as `0` and `u32::MAX`.

The most important property checked for Exercise 1 is that both implementations always agree: the recursive solution and the Morris solution must return the same result on every tested tree.

For the maximum path sum problem, I added tests for:

- a standard non-trivial tree with several leaves;

- a single-node tree and a skewed tree, where no leaf-to-leaf path exists and the correct result is `None`;
- a case where the best path is entirely inside a subtree and does not pass through the root;
- large key values, to verify that the implementation correctly returns a `u64` result without losing information.

Overall, the current test suite gives good coverage of both normal and corner cases, and it helped verify that the implementations behave correctly on valid inputs, invalid BSTs, degenerate trees, and numerical boundary values.